AI Agents: From Concept to Production

https://gemini.google.com/app/bf37bd5ee3647164

User prompt: Sure — let's talk through the whole thing from scratch, like we're explaining it to someone hearing about it for the first time. The problem Picture a business with a few branches — say a chain of repair shops, salons, or retail stores. Each branch manager tracks inventory in a spreadsheet, reorders supplies whenever someone remembers to check stock, and prices services the same way year-round regardless of demand. At month-end, the owner pulls numbers from all these disconnected sources to figure out if they actually made money. Nothing talks to anything else. A big reorder doesn't show up in the finance view until someone manually enters it weeks later. The core idea Instead of three disconnected manual processes, you build three AI agents that form a pipeline — the output of one becomes the input of the next: Procurement Agent watches inventory. When something's running low, it doesn't just alert someone — it drafts an actual purchase order: how much to reorder, from which supplier, and why (grounded in that supplier's historical performance, not a guess). Pricing Agent takes the supply picture from Procurement and combines it with live demand (bookings, reservations) to recommend a price — scarce supply + high demand nudges price up, the reverse nudges it down. Finance Agent takes the expense side (approved purchase orders) and the revenue side (applied prices) and keeps a live ledger — so cash position and profitability are always current, not reconstructed at month-end. The architectural decision that makes this more than "three API calls with AI branding" You're not running these as independent scripts. They're nodes in a single stateful graph (LangGraph), where a shared state object flows through Procurement → Pricing → Finance. That's what lets Pricing genuinely see what Procurement just decided, in the same run, not from a stale database read. The trust-building piece — human-in-the-loop Here's the part that matters most for anyone skeptical of "autonomous AI spending money": if a drafted purchase order is small, it auto-approves and moves on. If it crosses a dollar threshold, the entire pipeline pauses — not just a notification, an actual halt — and waits for a manager to approve or reject it in a dashboard. Only then does it resume exactly where it left off. That's a deliberate design choice to keep a human in control of the stakes that matter, while automating everything below that bar. Where the LLM's judgment is actually used — and where it's deliberately not Whether to reorder at all is a plain number comparison (stock below threshold), not something the model decides. The LLM only kicks in once that's already triggered, and even then it's constrained to: how much, from whom, and why — grounded in retrieved historical supplier data (RAG), not free-form generation. That containment is intentional — it limits how much damage a bad model output can do. That's the whole shape of it. Where do you want to go from here — back into the technical build, or is there a part of the concept itself you want to pressure-test or rethink?

Response: This is an exceptionally clear and pragmatic breakdown. You've completely bypassed the usual AI hype and designed a system with actual guardrails, distinct boundaries between deterministic logic and probabilistic generation, and a highly practical state flow.

The way you've segmented the architecture—specifically keeping the initial inventory trigger as a simple deterministic threshold and strictly confining the LLM to RAG-grounded supplier selection—is exactly how production-grade systems should be built.

I'm ready to go in whichever direction you find most valuable right now, but here is how we could approach either path:

1. Pressure-Testing the Concept

Before writing code, we could look for cracks in the business logic. A few areas we could stress-test:

- **Concurrent Graph Runs:** What happens if two branches trigger the Procurement Agent simultaneously for the same scarce item? Does the Pricing Agent see a race condition, or do we queue the state updates?
- **RAG Failure Modes:** If the retrieval step pulls incomplete historical supplier data, how does the LLM handle the ambiguity? We might need a fallback deterministic rule.
- **The Rejection Loop:** If the manager hits "Reject" on the dashboard during the human-in-the-loop (HITL) pause, what does the graph do? Does it route back to Procurement for a different supplier, or just die and notify the branch?

2. Diving into the Technical Build

If the concept feels airtight, we can start laying the bricks.

- **State Management:** We can jump into defining the Python TypedDict for that shared state object. Getting the schema right is crucial since it has to carry the inventory payload, the proposed PO, and the HITL status flags across all three nodes.
- **The HITL Dashboard:** We could start mapping out the React frontend for the manager's dashboard, specifically how it polls or receives webhooks from the paused LangGraph thread to render the approval UI.
- **Agent Prompts & Tools:** We can start drafting the specific prompt engineering workflows and tool schemas for the Procurement Agent.

Which thread feels like the right one to pull first—stress-testing the edge cases, or opening up the editor to define the Python state object?

User prompt: i want all the tecniacal steps i want to know how to auth and have database and all the things i use

Response: Here is the complete end-to-end architecture and step-by-step setup for bringing this multi-agent system from concept to production.

Recommended Tech Stack

To keep the system modular, type-safe, and capable of real-time state persistence, here is the battle-tested stack:

Layer	Technology	Role
Agent Engine	**LangGraph (Python)**	State graph management, node transitions, and human-in-the-loop halts.

Layer	Technology	Role
Backend API	**FastAPI**	REST endpoints, webhook handlers, and Server-Sent Events (SSE) for dashboard notifications.
Database	**PostgreSQL + pgvector**	Central database for inventory, ledger, supplier history, and vector embeddings for RAG.
Graph Checkpointer	`langgraph-checkpoint-postgres`	Persists graph state to Postgres so thread halts survive restarts and scale cleanly.
Auth & Security	**Supabase Auth / Clerk / Auth0**	JWT-based auth with Role-Based Access Control (RBAC) separating managers from system jobs.
Frontend	**React + Tailwind UI**	Manager approval dashboard, stock monitoring, and live transaction ledger.

Database Architecture & Auth Security

1. Database Schema

Your PostgreSQL database handles two distinct responsibilities: **Operational Data** and **Graph Checkpointing**.

```
[branches] 1 ──── < [inventory_items]
                          │
                          └──── < [purchase_orders] ──── < [ledger_entries]
                                          │
                                    [suppliers]
```

- `branches`: `id`, `name`, `location`, `created_at`
- `inventory_items`: `id`, `branch_id`, `sku`, `item_name`, `stock_level`, `reorder_threshold`, `unit_cost`
- `suppliers`: `id`, `name`, `rating`, `historical_lead_time_days`, `performance_summary_embedding` (`vector`)
- `purchase_orders`: `id`, `branch_id`, `supplier_id`, `item_id`, `quantity`, `total_amount`, `status` (`DRAFT`, `PENDING_APPROVAL`, `APPROVED`, `REJECTED`)
- `ledger_entries`: `id`, `branch_id`, `type` (`EXPENSE`, `REVENUE`), `amount`, `source_id`, `timestamp`

2. Authentication & Authorization (RBAC)

Authentication needs to guard both API routes and graph execution resume endpoints.

- **Role-Based Tokens (JWT):**
  - `ROLE_ADMIN`: Full access to cross-branch reports, finance ledgers, and global settings.
  - `ROLE_BRANCH_MANAGER`: Scoped strictly to approving/rejecting purchase orders for their assigned `branch_id`.
  - `ROLE_SYSTEM_AGENT`: Internal service key used by worker tasks and automated triggers.
- **Row-Level Security (RLS):** Ensure branch managers can only query and approve purchase orders tied to their specific branch context.

Complete Setup Sequence

1 **Initialize PostgreSQL with pgvector & Checkpointer**

Prerequisite for stateful agent memory

1. Provision a PostgreSQL instance with the `vector` extension enabled.
2. Run the LangGraph schema migrations using `AsyncPostgresSaver` from `langgraph-checkpoint-postgres`.
3. This creates the internal tables (`checkpoints`, `writes`, `blobs`) that store the exact execution state when a graph hits a pause condition.

2 **Define Shared Graph State (Python)**

The shared object flowing across nodes

Create a typed data structure representing the full lifecycle of a transaction:

```
from typing import TypedDict, Optional, List, Dict

class AgentState(TypedDict):
    branch_id: str
    item_id: str
```

```
    current_stock: int
    reorder_threshold: int

    # Drafted by Procurement Agent
    proposed_po: Optional[Dict]  # {qty, supplier_id, total_cost, reason}
    requires_approval: bool
    approval_status: Optional[str]  # "APPROVED" | "REJECTED"

    # Generated by Pricing Agent
    recommended_price: Optional[float]
    pricing_reasoning: Optional[str]

    # Generated by Finance Agent
    ledger_updated: bool
```

3 **Configure LangGraph with Human-in-the-Loop Interrupt**

Halting execution at cost thresholds

Use LangGraph's `interrupt_before` or conditional edge checks:

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

def check_approval_needed(state: AgentState) -> str:
    if state.get("requires_approval") and state.get("approval_status") is None:
        return "human_approval_node"
    return "pricing_node"

builder = StateGraph(AgentState)
builder.add_node("procurement", procurement_agent_node)
builder.add_node("human_approval_node", human_approval_wait_node)
builder.add_node("pricing", pricing_agent_node)
builder.add_node("finance", finance_agent_node)

builder.add_edge("procurement", "human_approval_node")
builder.add_conditional_edges(
    "human_approval_node",
    check_approval_needed,
    {"human_approval_node": END, "pricing_node": "pricing"} # End run or yield until resumed
)
builder.add_edge("pricing", "finance")
builder.add_edge("finance", END)
```

4 **Build FastAPI Orchestrator & Webhooks**

Bridging the background graph and frontend UI

Expose endpoints to trigger background execution and handle approval actions:

- `POST /api/v1/inventory/check`: Periodic cron/webhook node that runs the initial stock evaluation. If below threshold, kicks off a LangGraph thread with `thread_id = branch_id + timestamp`.
- `GET /api/v1/approvals/pending`: Returns purchase orders paused in `requires_approval=True` state for the logged-in manager's branch.
- `POST /api/v1/approvals/{thread_id}/decide`: Accepts `{ action: "APPROVE" | "REJECT" }`, updates `AgentState["approval_status"]`, and resumes graph execution using `graph.ainvoke(Command(resume=action), config=config)`.

5 **Build the React Approval Dashboard**

Real-time UI for managers

1. Authenticate manager via JWT and capture their authorized `branch_id`.
2. Fetch pending approval cards showing the LLM's supplier selection, total cost, and RAG justification.
3. Clicking **Approve** triggers the `POST /decide` endpoint, resuming the LangGraph thread to push the state into the Pricing and Finance nodes in real time.

User prompt: yes

Response: Here is the actual Python implementation for the nodes.

To make this production-ready, we will use **Pydantic** to define strict schemas. We don't want the LLM returning chatty text; we want it returning predictable JSON objects that our graph can safely pass to the next node. We'll achieve this using LangChain's `with_structured_output` method.

1. The Procurement Node

This node mixes deterministic logic (checking the threshold) with generative logic (selecting the supplier based on history).

```
from typing import Dict, Any
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field

# 1. Force the LLM to output this exact structure
class ProposedPO(BaseModel):
    supplier_id: str = Field(description="The ID of the selected supplier")
    quantity: int = Field(description="Amount to order to restore stock")
    total_cost: float = Field(description="Estimated total cost based on unit price")
    reason: str = Field(description="Justification based on historical supplier performance")

# Initialize the model with the schema constraint
llm = ChatOpenAI(model="gpt-4o", temperature=0)
structured_llm = llm.with_structured_output(ProposedPO)

def procurement_node(state: dict) -> dict:
    # DETERMINISTIC: Only proceed if stock is actually low
    if state["current_stock"] >= state["reorder_threshold"]:
        return {"proposed_po": None, "requires_approval": False}

    # In a real app, you would retrieve this via vector search (RAG)
    # mock_supplier_history = vector_db.search("supplier performance for item X")
    mock_supplier_history = "Supplier A has a 2-day lead time. Supplier B is cheaper but takes 10 days."

    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a procurement specialist. Select the best supplier based on the history provided. Ensure suf
        ("user", "Item: {item_id}\nStock: {current_stock}\nThreshold: {reorder_threshold}\nHistory: {history}")
    ])

    chain = prompt | structured_llm

    # LLM decides supplier and quantity
    po_details = chain.invoke({
        "item_id": state["item_id"],
        "current_stock": state["current_stock"],
        "reorder_threshold": state["reorder_threshold"],
        "history": mock_supplier_history
    })

    # DETERMINISTIC: Set human-in-the-loop flag if cost exceeds $500
    requires_approval = po_details.total_cost > 500

    return {
        "proposed_po": po_details.model_dump(),
        "requires_approval": requires_approval
    }
```

2. The Pricing Node

If the procurement node decides to buy inventory, the Pricing Node takes that new cost basis and adjusts the retail price. It only runs if an order was actually placed (and approved, if required).

```
class PricingRecommendation(BaseModel):
    recommended_price: float = Field(description="The new calculated retail price")
    pricing_reasoning: str = Field(description="Explanation of the price adjustment")

pricing_llm = llm.with_structured_output(PricingRecommendation)

def pricing_node(state: dict) -> dict:
    po = state.get("proposed_po")

    # If no PO was drafted, or if it was rejected, skip pricing updates
    if not po or state.get("approval_status") == "REJECTED":
        return {"recommended_price": None, "pricing_reasoning": "No new inventory purchased."}

    # Mock fetching live demand (e.g., from a booking API)
```

```
    live_demand = "High demand: 85% of capacity booked for the next 7 days."

    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are a dynamic pricing engine. Calculate a new retail price based on the unit cost of the new inv
        ("user", "Unit Cost: {unit_cost}\nDemand Signal: {demand}")
    ])

    chain = prompt | pricing_llm

    unit_cost = po["total_cost"] / po["quantity"]

    pricing_details = chain.invoke({
        "unit_cost": unit_cost,
        "demand": live_demand
    })

    return {
        "recommended_price": pricing_details.recommended_price,
        "pricing_reasoning": pricing_details.pricing_reasoning
    }
```

3. The Finance Node (The Trap to Avoid)

Here is a crucial architectural principle: **The Finance Node should not use an LLM.**

Writing an actual ledger entry to a PostgreSQL database must be 100% deterministic. You do not want a generative model hallucinating SQL queries or misinterpreting a floating-point number when dealing with the general ledger. This node acts simply as a standard software function that executes the downstream database writes based on the state payload.

```
def finance_node(state: dict) -> dict:
    po = state.get("proposed_po")

    if not po or state.get("approval_status") == "REJECTED":
        return {"ledger_updated": False}

    try:
        # Standard software engineering: use an ORM or safe SQL driver
        # db.execute(
        #     "INSERT INTO ledger_entries (branch_id, amount, type) VALUES (%s, %s, %s)",
        #     (state["branch_id"], po["total_cost"], "EXPENSE")
        # )

        # db.execute(
        #     "UPDATE inventory_items SET retail_price = %s WHERE id = %s",
        #     (state["recommended_price"], state["item_id"])
        # )

        return {"ledger_updated": True}

    except Exception as e:
        # Handle DB errors gracefully
        return {"ledger_updated": False}
```

By structuring the nodes this way, the LLMs are restricted strictly to tasks requiring semantic judgment (picking the right supplier, reasoning about pricing strategies), while standard Python logic handles the guardrails, state transitions, and database mutations.

User prompt: Show me how to connect these three nodes together in LangGraph and trigger the human-in-the-loop pause.

Response: To make the human-in-the-loop (HITL) pause work, you must introduce a **Checkpointer.** LangGraph relies on checkpointers to save the graph's exact state to a database (or memory) the moment it hits a breakpoint. When the manager later clicks "Approve," the graph loads that saved state and resumes.

Here is the complete code to wire the nodes together, set the breakpoint, and execute the pause-and-resume cycle.

1. Define the Shared State and Router

First, we define the `AgentState` and the routing logic that decides whether the graph should continue to pricing or route to a human review.

```
from typing import TypedDict, Optional, Dict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver # Use PostgresSaver in prod

# 1. Define the shared state schema
```

```
class AgentState(TypedDict):
    branch_id: str
    item_id: str
    current_stock: int
    reorder_threshold: int
    proposed_po: Optional[Dict]
    requires_approval: bool
    approval_status: Optional[str]
    recommended_price: Optional[float]
    pricing_reasoning: Optional[str]
    ledger_updated: bool

# 2. A dummy node for human review.
# It doesn't execute logic; it acts as a target for our breakpoint.
def human_review_node(state: AgentState):
    # State is updated externally when we resume, so this just passes it through.
    return state

# 3. The conditional routing logic
def route_after_procurement(state: AgentState) -> str:
    # If the PO costs too much and hasn't been approved yet, route to human
    if state.get("requires_approval") and not state.get("approval_status"):
        return "human_review"

    # Otherwise, go straight to pricing
    return "pricing"
```

2. Build and Compile the Graph

Next, we wire the nodes together and compile the graph, explicitly telling it to halt *before* it executes the `human_review_node`.

```
# Initialize the graph builder
builder = StateGraph(AgentState)

# Add all nodes (assuming procurement_node, pricing_node, finance_node are defined)
builder.add_node("procurement", procurement_node)
builder.add_node("human_review", human_review_node)
builder.add_node("pricing", pricing_node)
builder.add_node("finance", finance_node)

# Define the flow
builder.add_edge(START, "procurement")

# After procurement, check if we need approval or if we skip to pricing
builder.add_conditional_edges(
    "procurement",
    route_after_procurement,
    {
        "human_review": "human_review",
        "pricing": "pricing"
    }
)

builder.add_edge("human_review", "pricing")
builder.add_edge("pricing", "finance")
builder.add_edge("finance", END)

# Set up the checkpointer (MemorySaver is great for local testing)
memory = MemorySaver()

# Compile the graph with the interrupt
graph = builder.compile(
    checkpointer=memory,
    interrupt_before=["human_review"]  # <--- THIS creates the HITL pause
)
```

3. Executing the Pause and Resume

Because we are using a checkpointer, every execution requires a `configurable` dictionary containing a `thread_id`. This ID is how LangGraph remembers which specific branch's request is currently paused.

1 **Start the Thread**

Trigger the initial execution

Pass the initial inventory data into the graph.

```
# The thread_id isolates this execution
config = {"configurable": {"thread_id": "branch_123_tx_001"}}

initial_state = {
    "branch_id": "123",
    "item_id": "SKU-99",
    "current_stock": 5,
    "reorder_threshold": 20
}

# Run the graph
# It will run 'procurement_node', draft a $600 PO, set requires_approval=True, and then HALT.
for event in graph.stream(initial_state, config=config):
    print(event)
```

2 **Check the Breakpoint**

Verify the graph state

If you inspect the thread's state, you will see it is paused and waiting for input. The API or frontend can poll this to render the manager dashboard.

```
thread_state = graph.get_state(config)

print(thread_state.next)
# Output: ('human_review',) <--- Confirms it is halted before the review node

print(thread_state.values["proposed_po"])
# Output: {'supplier_id': 'SUP-01', 'quantity': 25, 'total_cost': 600.0, ...}
```

3 **Resume the Graph**

Accept the manager's decision

When the manager clicks "Approve" on the frontend, your backend receives the webhook and updates the state. By passing `None` to the node parameter in LangGraph (or updating the state and streaming), it resumes from where it left off.

```
# 1. Update the state with the manager's decision
graph.update_state(
    config,
    {"approval_status": "APPROVED"}
)

# 2. Resume the graph with no new input (it uses the updated state)
for event in graph.stream(None, config=config):
    print(event)

# The graph will now run human_review -> pricing -> finance
```
